

基于大模型的酒店客房服务智能体系统

客房服务 Agent —BRD 对齐项目大纲

客房服务 Agent 小组

2026 年 6 月 9 日

目录

1	项目现状总览	3
1.1	我们做了什么	3
1.2	6 个已实现的工具函数	3
1.3	当前架构图	3
1.4	BRD 对齐度评估	3
2	业务需求总览 (BRD 摘要)	4
2.1	客房服务 Agent 负责的 4 个意图	4
2.2	风险等级总览	4
2.3	核心槽位定义	4
3	目标架构: BRD 对齐后的完整流水线	5
3.1	标准化输出格式	6
4	三阶段实施计划	7
4.1	第一阶段: 配置落地 + 输出标准化 (预计 8.5 小时)	7
4.2	第二阶段: 校验链路 (预计 10 小时)	7
4.3	第三阶段: 工具对齐 + 测试 (预计 9 小时)	7
4.4	总时间线	8
5	技术栈与工具链	9
6	验收标准 (BRD AC1-AC5)	9
7	文件结构总览	10

8	手把手实操指南：从现状到 BRD 对齐	11
8.1	Day 1: 配置表落地（一）——General + IntentDefinitions	11
8.2	Day 2:配置表落地(二)——SlotDefinitions + CapabilityMatrix + RiskCon- trol	14
8.3	Day 3: 数据模型定义（models.py）	17
8.4	Day 4: 重写 System Prompt + 改造 Chatbot 节点	20
8.5	Day 5: 槽位校验器（slot_validator.py）	22
8.6	Day 6: 能力 Gating + 风控红线	22
8.7	Day 7: NeedClarify + ResponseFormatter + Trace	24
8.8	Day 8: 工具函数对齐 + 补齐缺失工具	25
8.9	Day 9: Locale + Entity 解析 + 验收测试	26
8.10	Day 10: 全链路联调 + Gradio 验收	26
8.11	Day 1-10 总览表	28
9	后续扩展方向	29

1 项目现状总览

1.1 我们做了什么

截至目前，客房服务 Agent 已有一个可运行的原型（MVP），核心组件如下：

组件	技术选型	状态
大模型	Qwen2.5:7B (Ollama 本地部署)	已就绪
AI 框架	LangGraph (图编排 + Tool Calling)	已就绪
知识库	Chroma + all-MiniLM-L6-v2 (RAG)	已就绪
会话记忆	MemorySaver (按房间号隔离)	已就绪
服务接口	FastAPI (REST API)	已就绪
测试界面	Gradio (ChatInterface)	已就绪
主路由	LLM 意图分类 → 5 路 Agent 分发	已就绪

1.2 6 个已实现的工具函数

1. `order_room_service(room_number, item, quantity)` — 客房送餐/饮品
2. `request_cleaning(room_number, time_preference)` — 预约清扫
3. `request_supplies(room_number, item, quantity)` — 补充消耗品
4. `report_maintenance(room_number, issue, urgency)` — 设备报修
5. `request_laundry(room_number, items, pickup_time)` — 洗衣服务
6. `set_wake_up_call(room_number, time)` — 叫醒服务

1.3 当前架构图

原型采用 LangGraph 构建的流水线图，节点依次为：内容安全护栏 → RAG 知识检索 → LLM 对话（含工具调用）→ 回复。

1.4 BRD 对齐度评估

根据 2026-06-09 的差距分析，当前代码对 BRD 的对齐度为约 **30%**。以下为关键差距：

BRD 要求	当前状态	差距
配置表落地	缺失	6 张配置表仅在文档中，代码未加载
槽位校验	缺失	未校验 request_type 枚举、duration 范围等
能力矩阵 Gat-ing	缺失	未检查 service 设备是否支持当前 intent
风控红线	缺失	3 个高风险 intent 直接执行，无二次确认
NeedClarify 机制	缺失	缺槽/歧义时无结构化澄清输出
结构化输出	缺失	仅回文本，无 result_type/decision_trace
工具完整性	缺 3 个	缺 delete_alarm / close_alarm / call_hotel

2 业务需求总览 (BRD 摘要)

2.1 客房服务 Agent 负责的 4 个意图

根据酒店 BRD 全表中的 IntentDefinitions，客房服务 Agent 负责以下意图（其余意图分别属于控制 Agent、咨询 Agent、点餐 Agent）：

ID	Intent L1	L2	L3	Required Slots
SVC_ROOM_001	ROOM_SERVICE	CREATE_REQUEST	DEFAULT	request_type
SVC_HK_001	HOUSEKEEPING	CREATE_REQUEST	DEFAULT	request_type
SVC_CALL_001	HOTEL_CALL	CREATE_REQUEST	DEFAULT	request_type
ALARM_001	ALARM	SETTINGS	DEFAULT	time, duration
ALARM_002	ALARM	DELETE	DEFAULT	label
ALARM_003	ALARM	CLOSE	DEFAULT	alarm_action

表 1: 客房服务 6 条意图定义（来自 IntentDefinitions.xlsx）

2.2 风险等级总览

2.3 核心槽位定义

Intent L1	风险等级	后处理要求
HOUSEKEEPING	高	必须二次确认；确认通过后才创建请求
HOTEL_CALL	高	必须二次确认
ROOM_SERVICE	高	必须二次确认；确认通过后才创建请求
ALARM	中	delete/close 必须二次确认；set 可直接执行

表 2: 客房服务意图风险等级（来自 BRD §7）

Slot	Min	Max	Enum	
request_type	—	—	room_service, housekeeping, hotel_call, workorder, amenity, other	三
priority	—	—	low, normal, high, urgent	三
alarm_action	—	—	set, delete, close	A
time	—	—	—	A
duration	1.0	10080.0	—	A
details	—	—	—	三
location	—	—	—	月

表 3: 客房服务核心槽位（来自 SlotDefinitions.xlsx）

3 目标架构：BRD 对齐后的完整流水线

改造后，LangGraph 图中的节点将扩展为以下 10 个节点：

顺序	节点
1	content_safety_filter (内容安全护栏)
2	locale_resolver (语言检测与回退)
3	rag_retrieve (知识库检索)
4	chatbot (LLM 意图识别 + 槽位提取，结构化 JSON 输出)
5	slot_validator (槽位 enum/range/required 校验)
6	entity_resolver (实体标准化与歧义检测)
7	capability_gate (能力矩阵检查)
8	risk_checker (风控红线 GR-01~10)
9	tools / execute (工具调用执行)
10	response_formatter + decision_trace (标准化输出)

每个节点都从 BRD 配置表中读取校验规则，确保所有决策可追溯。

3.1 标准化输出格式

系统最终输出统一为以下 JSON 结构:

```
{
  "final_intent": { "L1": "HOUSEKEEPING", "L2": "CREATE_REQUEST",
                    "L3": "DEFAULT", "ID": "SVC_HK_001" },
  "final_slots": { "request_type": "housekeeping", "location": "301",
                   "priority": "normal", "details": "现在打扫" },
  "resolved_entities": { "room": "301" },
  "result_type": "execute",
  "decision_trace": [
    { "step": "slot_validator", "result": "pass", "details": "..." },
    { "step": "capability_gate", "result": "pass" },
    { "step": "risk_checker", "result": "risky", "action": "confirm" }
  ]
}
```

其中 `result_type` 为三元枚举:

- "execute" 一校验全部通过, 可执行
- "need_clarify" 一缺槽/歧义/风险待确认, 需追问
- "reject" 一超出能力范围, 礼貌拒绝

4 三阶段实施计划

4.1 第一阶段：配置落地 + 输出标准化（预计 8.5 小时）

目标：让代码“读懂 BRD”，结束纯自由文本回复，输出标准化 JSON。

#	任务	产出
P0.1	新建 config/ 目录	6 张 BRD 配置表 JSON 化：general.json、intent_definitions.json、slot_definitions.json、capability_matrix.json、risk_control.json
P0.2	新建 models.py	数据模型：IntentCandidate、Slot、DecisionTrace、NeedClarify、FinalOutput
P0.3	重写 system_prompt.txt	注入 6 条 intent 定义 +required/optional 槽位，要求 LLM 输出结构化 JSON
P0.4	改造 chatbot_node	LLM 输出从自然语言改为结构化 JSON {intents, slots, entities}
P0.5	新建 response_formatter.py	标准化输出 {result_type, final_intent, ..., decision_trace}

4.2 第二阶段：校验链路（预计 10 小时）

目标：新增 5 个校验节点，让系统自动拒绝不合规请求。

4.3 第三阶段：工具对齐 + 测试（预计 9 小时）

目标：补齐缺失工具，通过 AC1-AC5 验收测试。

#	任务	产出
P1.1	slot_validator 节点	request_type 枚举校验、duration 范围 [1,10080]、required 槽位检查
P1.2	capability_gate 节点	查 capability_matrix.json, block 不支持的 intent
P1.3	risk_checker 节点	intent 级风险红线 + 全局 GR-01~10 检查
P1.4	clarify_builder 模块	14 个 reason_code 枚举 + candidates 标准结构 + clarify_slot
P1.5	改造 build_graph()	将 5 个新节点插入现有 Lang-Graph 图

#	任务	产出
P2.1	改造全部工具函数	加 request_type 参数, 返回结构化 trace
P2.2	新增 3 个缺失工具	delete_alarm(ALARM_002) / close_alarm(ALARM_003) / call_hotel(HOTEL_CALL)
P2.3	locale_resolver + entity_resolver	locale 检测回退 + 实体标准化歧义检测
P2.4	验收测试	AC1~AC5 测试用例: 枚举闭环/能力 Gating/槽位 clamp/required 槽澄清/实体可追溯

4.4 总时间线

阶段	工时	产出状态
第一阶段	~8.5h	配置表就绪 + 结构化输出
第二阶段	~10h	校验链路完整
第三阶段	~9h	工具完整 + 验收通过
总计	~27.5h	BRD 完全对齐

5 技术栈与工具链

层级	技术	用途
编程语言	Python 3.12	全部后端逻辑
本地模型	Ollama + Qwen2.5:7B	意图识别与槽位提取
AI 框架	LangGraph + LangChain	图编排、Tool Calling、会话记忆
向量数据库	Chroma	RAG 知识检索
Embedding	all-MiniLM-L6-v2	文本向量化
服务框架	FastAPI + Uvicorn	REST API 服务
测试界面	Gradio	交互式调试

6 验收标准（BRD AC1-AC5）

AC1: 枚举值闭环: 输出枚举值全部在 General/Slot enum 内，否则必须回退/澄清/拒绝且 trace 可定位。

AC2: 能力矩阵严格生效: 不支持的 intent_L1 不得 execute。

AC3: 槽位范围与 clamp 生效: duration 越界自动 clamp 到 [1, 10080]，并在 trace 标识。

AC4: 缺失 required 槽位必进入 need_clarify: 给出 reason_code + clarify_slot + candidates。

AC5: 实体/动作命中可追溯: trace 包含命中 variants、来源词表、canonical。

7 文件结构总览

改造完成后的完整文件结构：

```

llm/
|-- config/                                <-- 新建：BRD配置表JSON化
|   |-- general.json                      <-- language/device_type/location/scope
|   |-- intent_definitions.json           <-- 6条客房服务intent
|   |-- slot_definitions.json             <-- 11个客房服务槽位
|   |-- capability_matrix.json            <-- service能力矩阵
|   +-- risk_control.json                 <-- intent级风险 + GR-01~10
|-- models.py                             <-- 新建：全部数据模型
|-- room_service_agent.py                 <-- 改造：核心图 + 新节点
|-- slot_validator.py                     <-- 新建：槽位校验
|-- capability_gate.py                     <-- 新建：能力Gating
|-- risk_checker.py                       <-- 新建：风控红线
|-- clarify_builder.py                    <-- 新建：NeedClarify构建
|-- response_formatter.py                 <-- 新建：标准化输出 + trace
|-- locale_resolver.py                    <-- 新建：locale检测
|-- entity_resolver.py                    <-- 新建：实体解析
|-- tools_api/
|   +-- mock_services.py                  <-- 改造：加request_type + trace返回
|-- prompts/
|   +-- system_prompt.txt                  <-- 重写：注入intent/slot定义
|-- knowledge/
|   +-- placeholder_info.txt               <-- 保留：酒店知识库
|-- main_router.py                        <-- 保留：5路分发路由
|-- server.py                             <-- 改造：结构化输出模型
+-- tests/
    +-- test_room_service.py               <-- 新建：AC1~AC5验收测试

```

8 手把手实操指南：从现状到 BRD 对齐

以下按每天可完成的粒度拆解，每一步都标注了：创建/修改哪个文件、放什么内容、为什么（对应哪条 BRD）、如何自测。

图例说明

- 新建 = 创建此前不存在的文件
- 改造 = 修改已有文件
- 保留 = 不动，照旧

8.1 Day 1：配置表落地（一）——General + IntentDefinitions

目标：把 BRD §8 中客房服务相关的枚举和意图定义从文档搬到 JSON 文件，代码能加载。

Step 1.1 —新建 config/general.json

路径：llm/config/general.json

来源：BRD §8.1 General 表（仅提取客房服务相关的枚举列）

写入以下 JSON 结构：

```
{
  "language": {
    "enum": ["zh-CN", "zh-GD", "en-US", "en-SG"],
    "default": "zh-CN",
    "description": { "zh-CN": "普通话", "zh-GD": "粤语",
      "en-US": "美式英语", "en-SG": "新加坡英语" }
  },
  "device_type": {
    "enum": ["light", "rgbw_light", "curtain", "air_conditioner",
      "television", "speaker", "service", "alarm"],
    "note": "service和alarm为BRD隐含类型，不在原General表中"
  },
  "location": {
    "enum": ["living_room", "bedroom", "bathroom", "balcony",
      "whole_house", "bedside", "corridor", "second_bedroom",
      "kitchen", "display_area", "conference_room", "study_room"]
  }
}
```

```
}  
}
```

为什么:BRD §10.1 要求所有枚举字段必须命中 General 集合,否则 need_clarify/reject。
自测: Python 中 `json.load(open("config/general.json"))` 能正常读取。

Step 1.2 —新建 config/intent_definitions.json

路径: llm/config/intent_definitions.json

来源: BRD §8.2 IntentDefinitions (仅6条客房服务相关的行)

写入以下 JSON (6 条 intent 完整定义):

```
{  
  "intents": [  
    {  
      "id": "SVC_ROOM_001",  
      "L1": "ROOM_SERVICE", "L2": "CREATE_REQUEST", "L3": "DEFAULT",  
      "device_type": "service",  
      "required": ["request_type"],  
      "optional": ["details", "location", "priority"],  
      "description": "amenity request / ticket (客房补给/服务请求)",  
      "risk_level": "high"  
    },  
    {  
      "id": "SVC_HK_001",  
      "L1": "HOUSEKEEPING", "L2": "CREATE_REQUEST", "L3": "DEFAULT",  
      "device_type": "service",  
      "required": ["request_type"],  
      "optional": ["details", "location", "priority"],  
      "description": "housekeeping request / ticket (客房清洁/打扫)",  
      "risk_level": "high"  
    },  
    {  
      "id": "SVC_CALL_001",  
      "L1": "HOTEL_CALL", "L2": "CREATE_REQUEST", "L3": "DEFAULT",  
      "device_type": "service",  
      "required": ["request_type"],  
      "optional": ["details", "location", "priority"],  
      "description": "call hotel / connect operator (呼叫前台/人工)",  
      "risk_level": "high"  
    }  
  ]  
}
```

```

    },
    {
      "id": "ALARM_001",
      "L1": "ALARM", "L2": "SETTINGS", "L3": "DEFAULT",
      "device_type": "alarm",
      "required": ["time", "duration"],
      "optional": ["label", "repeat"],
      "description": "set alarm/timer (设定叫醒/闹钟)",
      "risk_level": "medium"
    },
    {
      "id": "ALARM_002",
      "L1": "ALARM", "L2": "DELETE", "L3": "DEFAULT",
      "device_type": "alarm",
      "required": ["label"],
      "optional": ["alarm_id"],
      "description": "delete alarm/timer (删除闹钟)",
      "risk_level": "medium",
      "require_confirm": true
    },
    {
      "id": "ALARM_003",
      "L1": "ALARM", "L2": "CLOSE", "L3": "DEFAULT",
      "device_type": "alarm",
      "required": ["alarm_action"],
      "optional": ["label"],
      "description": "stop alarm/timer (关闭闹钟)",
      "risk_level": "medium",
      "require_confirm": true
    }
  ]
}

```

为什么：BRD §9 步骤 3-5 要求基于 IntentDefinitions 做候选意图整理和裁决。自测：Python 中能按 L1 查询到对应 intent 的 required slots 列表。

8.2 Day 2: 配置表落地（二）——SlotDefinitions + CapabilityMatrix + RiskControl

目标：完成剩余 3 张配置表的 JSON 化，配置层全部就绪。

Step 2.1 —新建 config/slot_definitions.json

路径：llm/config/slot_definitions.json

来源：BRD §8.3 SlotDefinitions（仅11个客房服务相关槽位）

```
{
  "slots": [
    { "id": "SL_001", "name": "alarm_action",
      "enum": ["set","delete","close"],
      "note": "闹钟动作" },
    { "id": "SL_002", "name": "alarm_id",
      "note": "闹钟唯一标识，用于精确删除/关闭" },
    { "id": "SL_015", "name": "details",
      "note": "服务请求详情文本；可抽取item/quantity/故障类型" },
    { "id": "SL_019", "name": "duration",
      "min": 1.0, "max": 10080.0,
      "note": "持续时间/延时（分钟）" },
    { "id": "SL_026", "name": "label",
      "note": "标签/名称（闹钟或请求备注）" },
    { "id": "SL_027", "name": "language",
      "enum": ["zh-CN","zh-GD","en-US","en-SG"],
      "note": "语种/方言标识" },
    { "id": "SL_028", "name": "location",
      "note": "位置/区域，按上下文映射到房间枚举" },
    { "id": "SL_034", "name": "priority",
      "enum": ["low","normal","high","urgent"],
      "default": "normal",
      "note": "请求优先级" },
    { "id": "SL_038", "name": "repeat",
      "note": "重复规则：once/daily/weekdays/weekends等" },
    { "id": "SL_039", "name": "request_type",
      "enum": ["room_service","housekeeping","hotel_call",
        "workorder","amenity","other"],
      "note": "服务请求类型；与业务路由对应" },
    { "id": "SL_046", "name": "time",
      "note": "时间点 HH:MM，支持am/pm、口语'早上7点'" }
```

```
]
}
```

Step 2.2 —新建 config/capability_matrix.json

路径：llm/config/capability_matrix.json

来源：BRD §8.4 CapabilityMatrix（仅service和alarm行）

```
{
  "service": {
    "supported_intents": [
      "HOTEL_CALL", "HOUSEKEEPING", "ROOM_SERVICE",
      "NEED_CLARIFY", "EXIT"
    ],
    "source_row": "CM_010"
  },
  "alarm": {
    "supported_intents": ["ALARM"],
    "source_row": "CM_013"
  }
}
```

为什么：BRD §10.2 要求能力矩阵严格生效——不支持的 intent 不得 execute。自测：查 capability_matrix["service"]["supported_intents"] 返回 5 个 intent。

Step 2.3 —新建 config/risk_control.json

路径：llm/config/risk_control.json

来源：BRD §7（intent级）+ §7.1（全局GR-01~10）

注意：ALARM的device_type是alarm不是service（CM_013）

```
{
  "intent_risk": {
    "HOUSEKEEPING": { "level": "high", "require_confirm": true },
    "HOTEL_CALL": { "level": "high", "require_confirm": true },
    "ROOM_SERVICE": { "level": "high", "require_confirm": true },
    "ALARM": { "level": "medium",
      "confirm_actions": ["delete", "close"] }
  },
  "global_rules": [
    { "id": "GR-01", "trigger": "ambiguous_entity",
```

```

    "action": "need_clarify", "desc": "目标对象不明确" },
  { "id": "GR-02", "trigger": "ambiguous_scope",
    "action": "need_clarify", "desc": "范围不明确" },
  { "id": "GR-03", "trigger": "irreversible_action",
    "action": "risky_action_need_confirm",
    "desc": "不可逆/高影响操作" },
  { "id": "GR-04", "trigger": "capability_unsupported",
    "action": "need_clarify", "desc": "能力不支持" },
  { "id": "GR-05", "trigger": "invalid_enum",
    "action": "need_clarify", "desc": "枚举非法" },
  { "id": "GR-06", "trigger": "parse_time_failed",
    "action": "need_clarify", "desc": "时间/时长解析失败" },
  { "id": "GR-07", "trigger": "intent_conflict",
    "action": "need_clarify", "desc": "规则冲突" },
  { "id": "GR-08", "trigger": "device_unavailable",
    "action": "need_clarify", "desc": "设备不可用" },
  { "id": "GR-09", "trigger": "low_confidence",
    "action": "need_clarify", "desc": "置信度不足" },
  { "id": "GR-10", "trigger": "missing_target",
    "action": "need_clarify", "desc": "缺少目标设备/位置" }
],
"clarify_priority": [
  "risky_action_need_confirm",
  "capability_unsupported", "device_unavailable",
  "intent_conflict",
  "ambiguous_entity", "entity_not_found",
  "missing_required_slot",
  "parse_time_failed", "parse_duration_failed",
  "low_confidence"
]
}

```

为什么：BRD §7.1 要求所有全局红线触发即 NeedClarify，§6.1.6 定义了优先级顺序。**自测：**能找到 `intent_risk["HOUSEKEEPING"]["require_confirm"] == true`。

8.3 Day 3: 数据模型定义 (models.py)

目标：把 BRD §5.2（输出契约）和 §6.1（NeedClarify 字段规范）转成 Python 数据类，后续所有模块统一使用。

Step 3.1 —新建 llm/models.py

需要定义以下 5 个核心数据模型：

1. **IntentCandidate** —候选意图：L1/L2/L3/ID + score
2. **Slot** —单个槽位：name + value + validation_status (valid/clamped/defaulted/invalid)
3. **NeedClarify** —澄清输出：reason_code (14 个枚举之一) + clarify_slot + candidates[] + prompt_key + confirm_action
4. **DecisionTrace** —追溯记录：step_name + rule_id + input + output + result + timestamp
5. **FinalOutput** —最终输出：final_intent + final_slots + resolved_entities + result_type (execute/need_clarity/reject) + decision_trace[]

关键代码结构：

```
from enum import Enum
from dataclasses import dataclass, field
from typing import Optional

class ResultType(str, Enum):
    EXECUTE = "execute"
    NEED_CLARIFY = "need_clarify"
    REJECT = "reject"

class ReasonCode(str, Enum):
    MISSING_REQUIRED_SLOT = "missing_required_slot"
    INVALID_ENUM = "invalid_enum"
    OUT_OF_RANGE_CLAMPED = "out_of_range_clamped"
    CAPABILITY_UNSUPPORTED = "capability_unsupported"
    AMBIGUOUS_ENTITY = "ambiguous_entity"
    ENTITY_NOT_FOUND = "entity_not_found"
    LOCALE_MISSING_DEFAULTED = "locale_missing_defaulted"
    INTENT_CONFLICT = "intent_conflict"
    LOW_CONFIDENCE = "low_confidence"
```

```

RISKY_ACTION_NEED_CONFIRM = "risky_action_need_confirm"
PARSE_TIME_FAILED = "parse_time_failed"
PARSE_DURATION_FAILED = "parse_duration_failed"
DEVICE_UNAVAILABLE = "device_unavailable"
OUT_OF_SCOPE = "out_of_scope"

@dataclass
class IntentCandidate:
    L1: str; L2: str; L3: str; id: str
    score: float = 1.0

@dataclass
class Slot:
    name: str; value: str
    status: str = "raw" # raw|valid|clamped|defaulted|invalid

@dataclass
class NeedClarify:
    reason_code: ReasonCode
    clarify_slot: str
    candidates: list = field(default_factory=list)
    prompt_key: Optional[str] = None
    target_intent: Optional[IntentCandidate] = None
    confirm_action: Optional[dict] = None

@dataclass
class DecisionTraceStep:
    step: str; result: str
    rule_id: Optional[str] = None
    input_data: dict = field(default_factory=dict)
    output_data: dict = field(default_factory=dict)

@dataclass
class FinalOutput:
    final_intent: Optional[IntentCandidate]
    final_slots: dict
    resolved_entities: dict
    result_type: ResultType
    decision_trace: list
    clarify_info: Optional[NeedClarify] = None

```

为什么:BRD §5.2 定义了必须输出的字段;§6.1 定义了 NeedClarify 必填字段;§11 要求所有决策可追溯。**自测:**`python -c "from models import FinalOutput, ResultType; print(ResultType.EXECUTE)"`

8.4 Day 4: 重写 System Prompt + 改造 Chatbot 节点

目标：让 LLM 不再自由发挥，而是按 BRD intent 定义输出结构化 JSON。

Step 4.1 — 重写 prompts/system_prompt.txt

新建内容必须包含：

1. **角色定义**：你是五星级酒店客房服务 Agent，仅处理 3 类服务请求（ROOM_SERVICE / HOUSEKEEPING / HOTEL_CALL）+ 1 类叫醒（ALARM）
2. **意图定义表**：直接嵌入 6 条 intent 的 L1/L2/L3/ID/required/optional
3. **槽位说明**：request_type 的 6 个枚举值含义、duration 范围 [1, 10080]、time 格式
4. **输出格式要求**：必须输出 JSON，格式为 {"intents": [...], "slots": {...}, "entities": {...}}
5. **工具调用铁律**：保留原有的 6 个工具使用规则，但加上”缺少 required 槽位时先追问再调工具”
6. **安全边界**：拒绝酒店服务以外的问题

为什么：BRD §9 步骤 2 要求 LLM 输出候选意图/槽位/实体；§10.4 要求系统输出可用于对话层的结构化信息。**自测**：用 Ollama 命令行测试 `ollama run qwen2.5:7b` 配合新 prompt，输入”送两瓶水到 301”，输出 JSON 含 {"L1": "ROOM_SERVICE", ...}。

Step 4.2 — 改造 chatbot_node (room_service_agent.py)

在现有 chatbot_node 中：

- **改**：system_prompt 加载新的 txt 文件
- **改**：LLM 调用时加 `format="json"`（qwen2.5 支持 Ollama 的结构化 JSON 模式）
- **改**：LLM 返回后增加 JSON 解析步骤，提取 intents/slots/entities
- **加**：解析失败时的 fallback（retry 一次，仍失败则走 need_clarify）
- **加**：将解析结果写入 State（新增 `raw_intents` / `raw_slots` 字段）

Step 4.3 扩展 State 定义 (room_service_agent.py)

在现有 State 中新增以下字段：

```
class State(TypedDict):
    messages: Annotated[list, add_messages]    # 已有
    context: str                               # 已有 (RAG)
    is_safe: str                               # 已有 (护栏)
    # 以下新增
    locale: str                                # Phase 0 新增
    raw_intents: list                           # chatbot输出
    raw_slots: dict                             # chatbot输出
    raw_entities: dict                         # chatbot输出
    final_intent: Optional[dict]                # 经校验的最终intent
    final_slots: dict                           # 经校验的最终slots
    result_type: str                           # execute/need_clarify/reject
    clarify_info: Optional[dict]                # NeedClarify详情
    decision_trace: list                       # 全链路追溯
    confirm_pending: bool                      # 是否等待二次确认
```

为什么：State 是 LangGraph 节点间传递数据的唯一方式，需要扩展以承载校验链路各节点的中间结果。**自测：**运行 `main_router.py`，输入“送两瓶水到 301”，日志中能看到 `raw_intents` 不为空。

8.5 Day 5: 槽位校验器 (slot_validator.py)

目标：对 chatbot 提取的槽位做 BRD §10.1 要求的 enum/range/required 校验。

Step 5.1 —新建 llm/slot_validator.py

实现以下 3 个校验函数：

1. `validate_enum(slot_name, value, slot_defs)` 一查 `slot_definitions.json` 中的 enum 列表，不匹配返回 `invalid_enum`
2. `validate_range(slot_name, value, slot_defs)` 一查 min/max，越界则 clamp 并标记 `out_of_range_clamped`
3. `check_required(intent_id, slots, intent_defs)` 一查 `intent_definitions.json` 中的 required 列表，缺失返回 `missing_required_slot`

槽位校验映射表（从 BRD §8.3 SlotDefinitions 摘取）：

槽位	校验类型	规则	失败 reason_code
request_type	enum	6 个枚举值	invalid_enum
priority	enum+default	4 个枚举值，默认 normal	invalid_enum
alarm_action	enum	set/delete/close	invalid_enum
duration	range	[1.0, 10080.0]	out_of_range_clamped
time	format	HH:MM 可解析，否则 parse_time_failed	parse_time_failed
language	enum+fallback	4 个枚举值，默认 zh-CN	locale_missing_defaulted

集成到图中：在 `build_graph()` 中注册 `slot_validator` 节点，放在 chatbot 之后。校验结果写入 State 的 `final_slots` 和 `decision_trace`。

为什么：BRD AC1 要求枚举值闭环、AC3 要求槽位范围与 clamp 生效、AC4 要求缺失 required 槽必进入 need_clarify。自测：输入“送东西到 301”（没说什么东西），应走到 need_clarify (missing_required_slot→details)。

8.6 Day 6: 能力 Gating + 风控红线

目标：实现 BRD §10.2 的能力矩阵 gating 和 §7 的风控红线检查。

Step 6.1 —新建 llm/capability_gate.py

- 加载 `capability_matrix.json`
- 根据 intent 的 `device_type` 查 `supported_intents`

- 不匹配 → `reason_code = capability_unsupported` → `need_clarify`
- 注意：ALARM 的 `device_type` 是 `alarm`，其他 3 个 intent 是 `service`

Step 6.2 —新建 `llm/risk_checker.py`

实现两级检查：

1. **Intent 级：**查 `risk_control.json` 中的 `intent_risk.HOUSEKEEPING/HOTEL_CALL/ROOM` 的 `require_confirm=true` → 首次触发输出 `risky_action_need_confirm + confirm_action` 结构
2. **全局 GR-01~10：**遍历 `global_rules` 数组，检查每条规则的 `trigger` 条件

二次确认流程：

- 高风险 intent 首次触发 → `result_type=need_clarify`, `reason_code=risky_action_need_confirm`, `confirm_action={对象/范围/参数摘要}`
- 用户回复“确认”/“好的” → `State.confirm_pending=True` → 放行到 `tools`
- 用户回复“取消” → 终止

集成到图中：`capability_gate` → `risk_checker` → `tools`（或分流到 `clarify_builder`）。

为什么：BRD §7 明确 3 个服务 intent 为高风险必须二次确认；AC2 要求能力矩阵严格生效。**自测：**输入“帮我打扫 301 房间”→ 首次输出 `need_clarify` 要求确认 → 回复“确认”→ 执行 `request_cleaning`。

8.7 Day 7: NeedClarify + ResponseFormatter + Trace

目标：实现 BRD §6.1 的标准澄清输出和 §5.2 的最终输出契约。

Step 7.1 —新建 llm/clarify_builder.py

实现 `build_clarify(reason_code, ...) → NeedClarify:`

- 按 BRD §6.1.5 的强约束：每个 `reason_code` 对应必填字段
- `missing_required_slot` → 输出 `clarify_slot=` 缺失字段名
- `ambiguous_entity` → 输出 `candidates` 候选列表
- `risky_action_need_confirm` → 输出 `confirm_action` 结构
- 按 §6.1.6 优先级排序：多个 `reason` 时选主 `reason_code`，其余写入 `additional_reasons`

Step 7.2 —新建 llm/response_formatter.py

实现 `format_response(state) → FinalOutput:`

```
if state.result_type == "execute":
    → 组装 final_intent + final_slots + resolved_entities
      + result_type="execute" + decision_trace
elif state.result_type == "need_clarify":
    → 嵌入 clarify_info (reason_code + clarify_slot + candidates)
elif state.result_type == "reject":
    → reason_code = out_of_scope + 可用能力提示
```

同时生成 `decision_trace`：遍历所有节点的校验结果，每条 `trace` 包含 `step` 名、`rule_id`、输入、输出、结果。

Step 7.3 —改造 build_graph()

将全部新节点插入现有图：

```
START → content_safety → locale_resolver → rag_retrieve
      → chatbot → slot_validator → entity_resolver
      → capability_gate → risk_checker → tools
      → response_formatter → END
```

每个校验节点旁路：失败 → `clarify_builder` → `response_formatter` → END。

为什么：BRD §5.2 要求输出 `final_intent/final_slots/resolved_entities/result_type/decision_trace`。AC5 要求实体/动作可追溯。自测：全链路测试——输入“送两瓶水到 301”输出完整 JSON 含所有字段。

8.8 Day 8: 工具函数对齐 + 补齐缺失工具

目标：改造现有 6 个工具 + 新增 3 个缺失工具，使工具层与 BRD intent 完全对应。

Step 8.1 —改造 tools_api/mock_services.py

每个工具函数增加 request_type 参数 + 返回值改为结构化：

改造前

```
def request_cleaning(room_number, time_preference="现在") -> str:
    return "已安排保洁..."
```

改造后

```
def request_cleaning(room_number, time_preference="现在",
                    request_type="housekeeping") -> dict:
    return {
        "status": "success",
        "intent_id": "SVC_HK_001",
        "request_type": request_type,
        "room_number": room_number,
        "message": "已安排保洁...",
        "trace": {"tool": "request_cleaning", "request_type": request_type}
    }
```

Step 8.2 —新增 3 个缺失工具

1. delete_alarm(label, alarm_id=None) 一对应 ALARM_002（删除闹钟），需二次确认
2. close_alarm(alarm_action="close", label=None) 一对应 ALARM_003（关闭闹钟），需二次确认
3. call_hotel(request_type="hotel_call") 一对应 HOTEL_CALL（呼叫人工），需二次确认

更新 ALL_TOOLS 列表，从 6 个变为 9 个。

工具 Intent 最终映射：

为什么：BRD IntentDefinitions 中 ALARM_002 和 ALARM_003 没有对应工具，HOTEL_CALL 也没有。自测：每个工具单独测试，确认返回结构化 dict 含 status 和 trace。

工具	Intent ID	request_type
request_supplies	SVC_ROOM_001	amenity
order_room_service	SVC_ROOM_001	room_service
request_cleaning	SVC_HK_001	housekeeping
report_maintenance	SVC_HK_001	workorder
request_laundry	SVC_HK_001	amenity
call_hotel	SVC_CALL_001	hotel_call
set_wake_up_call	ALARM_001	—(alarm_action=set)
delete_alarm	ALARM_002	—(alarm_action=delete)
close_alarm	ALARM_003	—(alarm_action=close)

8.9 Day 9: Locale + Entity 解析 + 验收测试

目标：补上 locale 和 entity 解析，编写 AC1-AC5 验收测试。

Step 9.1 一新建 llm/locale_resolver.py

- 检查用户输入中的 locale 信号（中/英文、粤语词汇）
- 默认回退到 zh-CN（从 general.json 读 default）
- 输出 reason_code=locale_missing_defaulted（trace 标记）

Step 9.2 一新建 llm/entity_resolver.py

- 客房服务的主要实体是**房间号**（如 301、1205）和**服务类型**（request_type）
- 房间号正则：\d{3,4}（3-4 位数字）
- 服务类型：由 chatbot 的 slot 提取已覆盖，此处做 confirm
- 歧义：多个房间号 → ambiguous_entity → 列出候选
- 缺失：没找到房间号 → 如 intent 需要 location 则 entity_not_found → need_clarify

Step 9.3 一新建 tests/test_room_service.py

为每条 AC 编写单元测试：

为什么：BRD §11 定义 AC1-AC5 为业务验收标准，必须全部通过才算对齐。

8.10 Day 10: 全链路联调 + Gradio 验收

目标：端到端跑通，用 Gradio 界面演示给团队看。

AC	测试用例	预期
AC1	”送东西到 301” request_type 缺失	need_clarify + invalid_enum 或 missing_slot
AC2	意图错误路由到不支持 device	capability_unsupported
AC3	”设置闹钟 50000 分钟” duration 越界	out_of_range_clamped → clamp 到 10080
AC4	”打扫”缺房间号	need_clarify + missing_required_slot (location)
AC5	”送水到 301”全路径	trace 含 intent 识别 → slot 校验 → 工具调用完整链路

Step 10.1 —全链路集成测试

1. 启动服务：python server.py 或 python main_router.py
2. 测试 6 条 intent 每条至少 1 个用例（execute 路径 + need_clarify 路径）
3. 测试二次确认流程：高风险 intent → 确认 → 执行
4. 测试 GR 红线触发：说”取消所有服务” → risky_action_need_confirm
5. 测试护栏：说”怎么黑 WiFi” → reject

Step 10.2 —Gradio UI 验收

启动 python room_service_agent.py，在浏览器用 Gradio 交互：

- 验证多轮对话记忆（同一 session_id 内追问）
- 验证退房清除（DELETE /api/sessions/{id}）
- 截图留档

Step 10.3 —更新 server.py 的 ChatResponse 模型

将 ChatResponse 从简单的 response: str 改为返回完整 FinalOutput 结构，让语音 Agent 和控制 Agent 能拿到结构化数据。

8.11 Day 1-10 总览表

Day	阶段	动作	产出文件
1	P0	新建 general.json + intent_definitions.json	2 个 JSON
2	P0	新建 slot_definitions.json + capability_matrix.json + risk_control.json	3 个 JSON
3	P0	新建 models.py (6 个数据类) + 扩展 State	models.py
4	P0	重写 system_prompt.txt + 改造 chatbot_node	prompt + age
5	P1	新建 slot_validator.py + 集成到图	slot_validator
6	P1	新建 capability_gate.py + risk_checker.py	2 个 py
7	P1	新建 clarify_builder.py + response_formatter.py + 改造图	2 个 py + age
8	P2	改造 6 个工具 + 新增 3 个工具	mock_services
9	P2	新建 locale_resolver.py + entity_resolver.py + tests/	3 个 py
10	P2	全链路联调 + Gradio 验收 + server.py 改造	测试通过

表 4: 10 天实操计划总览

9 后续扩展方向

1. **与点餐 Agent 边界协商:** `order_room_service` 的”送餐”功能是否需要移交给点餐 Agent, 或保留在客房 Agent 作为”饮品/零食配送”。
2. **MemorySaver → SqliteSaver 升级:** 当前 MemorySaver 重启即丢失, 生产环境需持久化。
3. **配置表热更新:** BRD 配置表支持灰度发布, 需实现配置文件变更后不重启生效。
4. **多语言支持:** 当前仅支持中文, BRD 定义了 zh-CN/zh-GD/en-US/en-SG 四种 locale。
5. **与真实酒店 PMS 系统对接:** 将 mock 工具函数替换为真实 API 调用。

本文档基于酒店 BRD 全表 (2026-01-23)
以及 2026-06-09 完成的 BRD 对齐差距分析编写。
原始 BRD 未做任何删改。